

내가 지금부터 문제를 풀건데, 완벽한 코드를 생성해줘

 ChatGPT gpt-5-5

좋아. 문제를 보내주면 다음 기준으로 코드를 작성할게.

- 문제를 정확히 분석
- 시간복잡도·공간복잡도 고려
- 모든 예외 케이스 검토
- 입력 형식과 출력 형식 100% 준수
- 가능한 한 최적의 알고리즘 사용
- 코드에 버그가 없도록 검증
- 필요하면 풀이 과정과 알고리즘 설명도 제공

문제를 그대로 붙여넣어 줘. (텍스트, 이미지 모두 가능)

가능하면 함께 알려줘:

- 사용 언어 (Python, C++, Java 등)
- 온라인 저지 사이트 (백준, 프로그래머스, SWEA, LeetCode 등)
- 시간 제한 / 메모리 제한 (알면)

문제를 보내면 바로 풀이해줄게.

우선 csv파일을 pandas로 불러와야해

 ChatGPT gpt-5-5

데이터 확인

```
print(df.head())"/>
```

엑셀에서 저장한 한글 **CSV**라면 **cp949** 또는 **euc-kr**를 시도

```
df = pd.read_csv("data.csv", encoding="cp949")/>
```

```
<CodeBlock language="python" content="# 탭(tab) 구분 df = pd.read_csv("data.csv", sep="\t")
```

세미콜론 구분

```
df = pd.read_csv("data.csv", sep=";")/>
```

CSV 불러오기

```
df = pd.read_csv("students.csv", encoding="utf-8")
```

데이터 확인

```
print(df.head()) print(df.info())/>
```

이제 CSV를 불러온 뒤 어떤 작업을 할지(전처리, 분석, 시각화, 머신러닝 등) 이어서 말해주면 그 다음 코드도 완성해줄게.

2026-06-11 12:08:20 You Asked 

```
#데이터 처리와 시각화를 위한 라이브러리
import pandas as pd
import matplotlib.pyplot as plt

#머신러닝 도구
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error,
mean_squared_error, r2_score
```

이 코드에서 어떻게 입력해야해? csv파일 이름은
cafe_daily_sales.csv야

 ChatGPT gpt-5-5

CSV 파일 이름이 cafe_daily_sales.csv 라면, 네가 올린 코드 아래에 다음 한 줄을 추가하면 돼.

```
python
#데이터 처리와 시각화를 위한 라이브러리
import pandas as pd
import matplotlib.pyplot as plt

#머신러닝 도구
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# CSV 파일 불러오기
df = pd.read_csv("cafe_daily_sales.csv")

# 데이터 확인
print(df.head())
```

만약 CSV 파일이 현재 파이썬 파일과 같은 폴더에 없다면 경로까지 적어줘야 해.

예시:

```
python

df = pd.read_csv("data/cafe_daily_sales.csv")
```

또는 Windows 경로:

```
python

df = pd.read_csv(r"C:\Users\user\Desktop\cafe_daily_sales.csv")
```

불러온 뒤에는 컬럼명을 먼저 확인하는 것이 좋아.

```
python

print(df.columns)
```

그러면 다음 단계인:

- 어떤 컬럼을 입력(X)으로 사용할지
- 어떤 컬럼을 예측값(y)으로 사용할지
- `train_test_split()`
- `LinearRegression()`

까지 정확하게 코드를 작성할 수 있어.

가능하면 `print(df.head())` 결과나 CSV의 컬럼명 목록을 보여줘. 그러면 바로 이어서 완성해줄게.



폴더에 등록이 안되었다해서 다운받은 csv를 먼저 옮겨야해



ChatGPT gpt-5-5

맞아. `pd.read_csv("cafe_daily_sales.csv")` 가 작동하려면 파이썬이 실행되는 작업 폴더(현재 작업 디렉터리)에 CSV 파일이 있거나, 파일의 전체 경로를 지정해야 해.

먼저 현재 파이썬이 어떤 폴더를 기준으로 찾고 있는지 확인해보자.

```
python
```

```
import os

print(os.getcwd())
```

예를 들어 출력이

```
text
```

```
C:\Users\rockchan\Documents\Python
```

이라면 `cafe_daily_sales.csv` 를 그 폴더 안에 넣어야 다음 코드가 작동해.

```
python
```

```
df = pd.read_csv("cafe_daily_sales.csv")
```

CSV를 옮기기 싫다면 다운로드 폴더에 있는 파일을 직접 지정할 수도 있어.

예시:

```
python
```

```
df = pd.read_csv(r"C:\Users\rockchan\Downloads\cafe_daily_sales.csv")
```

(`r` 을 앞에 붙이면 Windows 경로의 `\` 를 그대로 사용할 수 있음)

어떤 환경을 쓰는지도 알려줘.

- VS Code
- Jupyter Notebook

- Google Colab
- PyCharm
- Anaconda/Jupyter

환경에 따라 CSV를 넣는 방법이 조금씩 달라.

2026-06-11 12:11:02

You Asked 

vs code

 ChatGPT gpt-5-5

VS Code라면 가장 쉬운 방법은 **CSV** 파일을 프로젝트 폴더 안으로 옮기는 것이야.

방법 1 (추천)

예를 들어 폴더 구조가 이렇게 되어 있다고 하자.

text

```
카페매출분석/  
|  
├── main.py  
└── cafe_daily_sales.csv
```

그러면 코드에서:

python

```
import pandas as pd  
  
df = pd.read_csv("cafe_daily_sales.csv")  
print(df.head())
```

만으로 바로 읽을 수 있어.

현재 프로젝트 폴더 확인하기

VS Code에서 왼쪽 탐색기(Explorer)를 보면:

text

```
카페매출분석
|—— main.py
```

처럼 보일 텐데,

다운로드한 `cafe_daily_sales.csv` 를 드래그해서 그 폴더 안에 넣으면 돼.

결과:

text

```
카페매출분석
|—— main.py
|—— cafe_daily_sales.csv
```

파일이 제대로 읽히는지 확인

python

```
import pandas as pd

df = pd.read_csv("cafe_daily_sales.csv")

print(df.head())
print(df.columns)
print(df.shape)
```

출력 예시:

text

```
   date    sales
0 2024-01-01   120
1 2024-01-02   135

Index(['date', 'sales'], dtype='object')
(365, 2)
```

만약 `FileNotFoundError` 가 뜬다면

현재 작업 폴더를 확인해:

python

```
import os

print(os.getcwd())
```

출력된 경로와 CSV 파일이 있는 경로가 같은지 확인하면 돼.

CSV를 읽은 다음에는 컬럼명 확인이 중요해.

```
python

print(df.columns.tolist())
```

결과를 보여주면, 그 컬럼에 맞춰서 `train_test_split` → `LinearRegression` → `성능평가` 까지 이어지는 코드를 작성해줄게.

2026-06-11 12:11:56

You Asked 

c폴더에 파일을 하나 만들어서 해야하는건가

 ChatGPT gpt-5-5

아니. **C** 드라이브(**C:**)에 새 폴더를 꼭 만들 필요는 없어.

VS Code에서 작업 중인 프로젝트 폴더 안에만 넣으면 돼.

예를 들어 네가 바탕화면에 `PythonStudy` 라는 폴더를 만들어서 VS Code로 열었다면:

```
text

바탕화면
├── PythonStudy
│   ├── main.py
│   └── cafe_daily_sales.csv
```

이렇게만 되어 있어도 충분해.

그리고 코드에서는:

```
python

df = pd.read_csv("cafe_daily_sales.csv")
```

를 사용하면 돼.

만약 아직 프로젝트 폴더 자체가 없다면:

- 1. 바탕화면에 새 폴더 생성 (예: 카페분석)
- 2. `cafe_daily_sales.csv` 를 그 폴더에 넣기
- 3. VS Code에서 파일 → 폴더 열기로 `카페분석` 폴더 열기
- 4. 그 안에 `main.py` 생성
- 5. 코드 작성

폴더 구조는 이렇게:

```
text

카페분석
├── main.py
└── cafe_daily_sales.csv
```

현재 VS Code 왼쪽 탐색기에 뭐가 보이는지 알려줘.

예를 들어:

```
text

EXPLORER
OPEN EDITORS
main.py
```

처럼 보이는지,

아니면

```
text

EXPLORER
카페분석
├── main.py
```

처럼 폴더 이름이 보이는지 알려주면 지금 상태에서 어떻게 해야 하는지 정확히 알려줄게.



좋아. 문제를 보내줘.

가능하면 아래 내용을 함께 알려주면 더 정확하게 풀 수 있어.

- 문제 설명
- 입력 형식
- 출력 형식
- 제공된 CSV 파일 설명(컬럼명 포함)
- 과제 요구사항
- 사용 가능한 라이브러리 제한 여부

문제를 보내면:

1. 요구사항 분석
2. 데이터 불러오기 코드
3. 전처리
4. 머신러닝 모델 학습
5. 성능 평가(MAE, MSE, RMSE, R^2 등)
6. 시각화 코드
7. 최종 제출용 완성 코드

순서대로 작성해줄게.

2026-06-11 12:15:05 You Asked

여기서 조건을 알려줄게 이 조건들에 맞는 코드를 한번 생성해줘

1. 전체 행 수와 열별 결측값 개수를 출력할거고
2. 중복된 행의개수를 확인하고 제거한다.
3. day_of_week 열의 앞뒤 공백을 제거하고, 고유값이 7개 요일만 남았는지 확인한다.
4. sale 열에 결측값이 있는 행을 제거한다. 예측 대상인 매출이 없는

날은 학습에 사용할수 없기 때문이다.
우선 여기까지



CSV 파일을 `cafe_daily_sales.csv` 로 불러온 뒤, 네가 제시한 1~4번 조건을 모두 수행하는 코드는 아래와 같아.

python

```
# 데이터 처리와 시각화를 위한 라이브러리
import pandas as pd
import matplotlib.pyplot as plt

# 머신러닝 도구
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# CSV 파일 불러오기
df = pd.read_csv("cafe_daily_sales.csv")

# =====
# 1. 전체 행 수와 열별 결측값 개수 출력
# =====

print("전체 행 수:", len(df))

print("\n열별 결측값 개수")
print(df.isnull().sum())

# =====
# 2. 중복된 행 개수 확인 및 제거
# =====

duplicate_count = df.duplicated().sum()

print("\n중복된 행 개수:", duplicate_count)

df = df.drop_duplicates()

print("중복 제거 후 행 수:", len(df))

# =====
# 3. day_of_week 열 앞뒤 공백 제거
```

```

# 고유값이 7개 요일인지 확인
# =====

df["day_of_week"] = df["day_of_week"].str.strip()

print("\nday_of_week 고유값")

print(sorted(df["day_of_week"].unique()))

print("고유값 개수:", df["day_of_week"].nunique())

# =====
# 4. sales 열 결측값 제거
# =====

before_rows = len(df)

df = df.dropna(subset=["sales"])

after_rows = len(df)

print("\nsales 결측값 제거 전 행 수:", before_rows)
print("sales 결측값 제거 후 행 수:", after_rows)
print("제거된 행 수:", before_rows - after_rows)

```

주의

네 조건에는 **sale 열** 이라고 적혀 있지만, 실제 CSV 컬럼명이 다음 둘 중 하나일 가능성이 높아.

```
python
```

```
sale
```

또는

```
python
```

```
sales
```

먼저 아래 코드를 실행해서 실제 컬럼명을 확인해.

```
python
```

```
print(df.columns.tolist())
```

만약 결과가

```
python
```

```
['date', 'day_of_week', 'sales', ...]
```

라면 위 코드 그대로 사용하면 되고,

```
python
```

```
['date', 'day_of_week', 'sale', ...]
```

라면

```
python
```

```
df = df.dropna(subset=["sales"])
```

를

```
python
```

```
df = df.dropna(subset=["sale"])
```

로 바꿔야 해.

다음 조건(인코딩, 변수 변환, train_test_split, LinearRegression 학습, 성능평가 등)을 보내주면 이어서 작성해줄게.

2026-06-11 12:18:53 You Asked 

자 다음 이코드에 이어서 조건을 말해줄께,
temperature 열의 결측값을 해당 열의 평균값으로 채운다.
foot_traffic 열에 IQR 기준 ($Q1 - 1.5 * IQR$ 미만 또는 $Q3 + 1.5 * IQR$ 초과)을 적용해 이상치가 있는행을 제거한다.
그리고나서 sales 기준으로 sales_level 열을 만든다.
열의 조건은 380만원 이상: 높음 / 300만원 이상 380만원 미만: 보통 / 300만원 미만: 낮음이야

 ChatGPT gpt-5-5

앞에서 작성한 코드에 이어서 아래 코드를 추가하면 돼.

```
python
```

```

# =====
# 5. temperature 결측값을 평균값으로 대체
# =====

temp_mean = df["temperature"].mean()

df["temperature"] = df["temperature"].fillna(temp_mean)

print("\ntemperature 결측값 처리 후")
print("temperature 결측값 개수:", df["temperature"].isnull().sum())

# =====
# 6. foot_traffic 이상치 제거 (IQR 기준)
# =====

Q1 = df["foot_traffic"].quantile(0.25)
Q3 = df["foot_traffic"].quantile(0.75)

IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

before_rows = len(df)

df = df[
    (df["foot_traffic"] >= lower_bound) &
    (df["foot_traffic"] <= upper_bound)
]

after_rows = len(df)

print("\nfoot_traffic 이상치 제거")
print("제거 전 행 수:", before_rows)
print("제거 후 행 수:", after_rows)
print("제거된 행 수:", before_rows - after_rows)

# =====
# 7. sales_level 열 생성
# =====

def classify_sales_level(sales):
    if sales >= 3800000:
        return "높음"
    elif sales >= 3000000:
        return "보통"
    else:

```

```

        return "낮음"

df["sales_level"] = df["sales"].apply(classify_sales_level)

print("\nsales_level 분포")
print(df["sales_level"].value_counts())

# 생성 결과 확인
print("\n데이터 확인")
print(df[["sales", "sales_level"]].head())

```

처리 내용 요약

1. temperature

- 평균값(mean)으로 결측치 대체

2. foot_traffic

- Q1(25%)
- Q3(75%)
- IQR = Q3 - Q1
- 하한 = Q1 - 1.5×IQR
- 상한 = Q3 + 1.5×IQR
- 범위를 벗어나는 행 제거

3. sales_level

- 3,800,000 이상 → "높음"
- 3,000,000 이상 ~ 3,800,000 미만 → "보통"
- 3,000,000 미만 → "낮음"

다만 실제 CSV의 sales 값이 "만원 단위 숫자(380, 300)"인지, "원 단위 숫자(3,800,000 / 3,000,000)"인지에 따라 기준이 달라져.

먼저 아래를 실행해서 확인해 보:

```
python
```

```
print(df["sales"].describe())
```

만약 값이 250 , 340 , 410 처럼 나온다면 기준은:

```
python
```

```
if sales >= 380:
```

처럼 바꿔야 하고,

값이 2500000 , 3400000 , 4100000 처럼 나온다면 지금 코드가 맞아.

2026-06-11 12:20:56

You Asked



여기에 마지막조건으로 정리가 끝난 DataFrame의 행 수와상위 5행을 출력할거야.

그리고 결측값두가지(sales는 행 제거 / temperature는 평균채우기)를 서로 다르게 처리한 이유를 적는다.

서로 다르게 처리한 이유는 따로 적어줘



ChatGPT gpt-5-5

앞 코드에 이어서 마지막으로 아래를 추가하면 돼.

python

```
# =====  
# 8. 최종 데이터 확인  
# =====  
  
print("\n최종 데이터 행 수:", len(df))  
  
print("\n최종 데이터 상위 5행")  
print(df.head())
```

결측값 처리 방법을 다르게 적용한 이유

1. sales 결측값은 행 제거

sales 는 모델이 예측해야 하는 목표 변수(타겟 변수) 이다.

매출 데이터가 없는 행은 실제 정답이 없는 데이터이므로 머신러닝 학습에 사용할 수 없다. 따라서 임의의 값으로 채우기보다 해당 행을 제거하는 것이 적절하다.

2. temperature 결측값은 평균값으로 대체

temperature 는 매출을 설명하는 독립 변수(입력 변수) 이다.

일부 온도 데이터가 누락되었더라도 해당 행의 매출 정보는 존재하므로 데이터를 버리면 학습에 사용할 수 있는 데이터가 감소한다. 따라서 데이터 손실을 최소화하기 위해 해당 열의 평균값으로 결측값을 대체하였다.

제출용 요약 문장

sales는 예측 대상인 목표 변수이므로 결측값이 있는 행은 학습에 사용할 수 없어 제거하였다. 반면 temperature는 입력 변수이므로 해당 행의 다른 정보와 매출 데이터는 활용 가능하다. 따라서 데이터 손실을 줄이기 위해 평균값으로 결측치를 대체하였다.

2026-06-11 12:25:03

You Asked



```
print("전체 행 수:", len(df))

print("\n열별 결측값 개수")
print(df.isnull().sum())

duplicate_count = df.duplicated().sum()

print("\n중복된 행 개수:", duplicate_count)

df = df.drop_duplicates()

print("중복 제거 후 행 수:", len(df))

df["day_of_week"] = df["day_of_week"].str.strip()

print("\nday_of_week 고유값")

print(sorted(df["day_of_week"].unique()))

print("고유값 개수:", df["day_of_week"].nunique())

before_rows = len(df)

df = df.dropna(subset=["sales"])

after_rows = len(df)
```



```

print("\nsales 결측값 제거 전 행 수:", before_rows)
print("sales 결측값 제거 후 행 수:", after_rows)
print("제거된 행 수:", before_rows - after_rows)

temp_mean = df["temperature"].mean()

df["temperature"] = df["temperature"].fillna(temp_mean)

print("\ntemperature 결측값 처리 후")
print("temperature 결측값 개수:", df["temperature"].isnull().sum())

# =====
# 6. foot_traffic 이상치 제거 (IQR 기준)
# =====

Q1 = df["foot_traffic"].quantile(0.25)
Q3 = df["foot_traffic"].quantile(0.75)

IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

before_rows = len(df)

df = df[
    (df["foot_traffic"] >= lower_bound) &
    (df["foot_traffic"] <= upper_bound)
]

after_rows = len(df)

print("\nfoot_traffic 이상치 제거")
print("제거 전 행 수:", before_rows)
print("제거 후 행 수:", after_rows)
print("제거된 행 수:", before_rows - after_rows)

# =====
# 7. sales_level 열 생성
# =====

```

```
def classify_sales_level(sales):
    if sales >= 38000000:
        return "높음"
    elif sales >= 30000000:
        return "보통"
    else:
        return "낮음"

df["sales_level"] = df["sales"].apply(classify_sales_level)

print("\nsales_level 분포")
print(df["sales_level"].value_counts())

# 생성 결과 확인
print("\n데이터 확인")
print(df[["sales", "sales_level"]].head())

print("\n최종 데이터 행 수:", len(df))
print("\n최종 데이터 상위 5행")
print(df.head())
```

내가 적어준 조건을 바탕으로 코드 생성을 도와줬는데, 중간중간 데이터를 확인하는 코드들만 같이 적어줬던걸 지우고, 이 조건들을 포함한 8번 조건의 상위 5행 출력까지로 코드를 수정해줘



중간 점검용 print() 들은 제거하고, 조건 수행 + 최종 행 수와 상위 **5**행 출력만 남기면 아래처럼 정리할 수 있어.

python

```
# 전체 행 수와 열별 결측값 개수 확인
print("전체 행 수:", len(df))
print("\n열별 결측값 개수")
print(df.isnull().sum())

# 중복 행 제거
df = df.drop_duplicates()

# day_of_week 열의 앞뒤 공백 제거
```

```

df["day_of_week"] = df["day_of_week"].str.strip()

# sales 결측값 제거
df = df.dropna(subset=["sales"])

# temperature 결측값을 평균값으로 대체
temp_mean = df["temperature"].mean()
df["temperature"] = df["temperature"].fillna(temp_mean)

# foot_traffic 이상치 제거 (IQR 기준)
Q1 = df["foot_traffic"].quantile(0.25)
Q3 = df["foot_traffic"].quantile(0.75)

IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

df = df[
    (df["foot_traffic"] >= lower_bound) &
    (df["foot_traffic"] <= upper_bound)
]

# sales_level 열 생성
def classify_sales_level(sales):
    if sales >= 3800000:
        return "높음"
    elif sales >= 3000000:
        return "보통"
    else:
        return "낮음"

df["sales_level"] = df["sales"].apply(classify_sales_level)

# 최종 데이터 확인
print("\n최종 데이터 행 수:", len(df))

print("\n최종 데이터 상위 5행")
print(df.head())

```

다만 교수님이나 과제 지침에서 "고유값이 7개 요일만 남았는지 확인한다", "중복된 행 개수를 확인한다"라고 명시했다면, 해당 결과를 출력하는 코드까지는 남겨두는 것이 더 안전할 수 있어. 지금 코드는 네 요청대로 중간 확인용 출력들을 제거한 버전이야.





그게 더 안전해 보여.

과제에서 "확인한다", "검증한다", "출력한다" 같은 표현이 있으면 결과를 보여주는 `print()` 를 남겨두는 것이 감점 가능성을 줄여줘.

현재 네 코드를 보면 다음 요구사항들을 모두 확인할 수 있어.

- 전체 행 수 출력
- 열별 결측값 개수 출력
- 중복 행 개수 출력
- 중복 제거 후 행 수 출력
- `day_of_week` 고유값 출력
- 고유값 개수 출력(7개인지 확인 가능)
- `sales` 결측값 제거 전후 비교
- `temperature` 결측값 처리 확인
- `foot_traffic` 이상치 제거 전후 비교
- `sales_level` 생성 결과 확인
- 최종 행 수 출력
- 최종 상위 5행 출력

특히 데이터 전처리 과제에서는 결과를 확인하는 출력문 자체가 수행 증거가 되기 때문에, 보고서나 제출 코드에서는 오히려 남겨두는 편이 좋다.

이후 머신러닝 단계로 넘어가면 보통

1. 범주형 변수 인코딩
2. X, y 분리
3. `train_test_split()`
4. LinearRegression 학습
5. MAE, MSE, RMSE, R^2 평가
6. 실제값 vs 예측값 시각화

순서로 진행하게 될 가능성이 높아. 다음 문제 조건을 주면 이어서 같이 맞춰보자. 🧡

2026-06-11 12:26:32 You Asked 

자 이제 다음 문제야. 앞에서 정리한 DataFrame을 사용해 매출 패턴을 분석화하는 문제고, 이해했으면 또 다시 조건을 부여해줄게

 ChatGPT gpt-5-5

이해했어.

현재 기준으로는 전처리가 완료된 **df** 를 그대로 사용해서 매출(**sales**) 패턴을 분석하고 시각화하는 단계로 넘어가는 거야.

다음 조건을 알려주면 그 조건에 맞춰서:

- 필요한 집계(**groupby**)
- 통계 분석
- 시각화(**matplotlib**)
- 그래프 제목, 축 이름
- 출력 형식

까지 포함해서 코드를 작성해줄게.

조건을 보내줘.

2026-06-11 12:28:39 You Asked 

우선 첫번째 요구사항: 집계 분석 부분이야

1. **day_of_week**별 **sales** 평균을 높은 순서로 정렬해 출력한다.
2. **promotion** 진행 여부별 **sales** 평균을 출력한다.
3. **sales_level**별 일수와 **foot_traffic** 평균을 출력한다.
4. **sales**가 높은 순서로 정렬해 상위 5일의 **date**, **day_of_week**, **promotion**, **sales**를 출력한다.

이 4가지 조건을 확인하고 앞에서 쓴 dataframe을 사용하여 집계 분석하는 코드를 생성해줘

앞에서 전처리가 끝난 `df` 를 그대로 사용한다는 가정하에, 아래 코드를 추가하면 4가지 집계 분석 요구사항을 모두 수행할 수 있어.

python

```
# =====
# 1. day_of_week별 sales 평균 (높은 순 정렬)
# =====

weekday_sales_mean = (
    df.groupby("day_of_week")["sales"]
      .mean()
      .sort_values(ascending=False)
)

print("요일별 평균 매출 (내림차순)")
print(weekday_sales_mean)

# =====
# 2. promotion 진행 여부별 sales 평균
# =====

promotion_sales_mean = (
    df.groupby("promotion")["sales"]
      .mean()
)

print("\n프로모션 진행 여부별 평균 매출")
print(promotion_sales_mean)

# =====
# 3. sales_level별 일수와 foot_traffic 평균
# =====

sales_level_summary = (
    df.groupby("sales_level")
      .agg(
          days=("sales_level", "count"),
          avg_foot_traffic=("foot_traffic", "mean")
      )
)

print("\nsales_level별 일수 및 평균 유동인구")
print(sales_level_summary)
```

```
# =====
# 4. 매출 상위 5일 정보 출력
# =====

top5_sales_days = (
    df.sort_values(by="sales", ascending=False)
      [["date", "day_of_week", "promotion", "sales"]]
      .head(5)
)

print("\n매출 상위 5일")
print(top5_sales_days)
```

각 요구사항과 코드 대응

요구사항	코드
day_of_week별 sales 평균을 높은 순으로 정렬	groupby() + mean() + sort_values()
promotion 진행 여부별 sales 평균	groupby("promotion")["sales"].mean()
sales_level별 일수와 foot_traffic 평균	agg(count, mean)
sales 상위 5일 출력	sort_values(by="sales", ascending=False).head(5)

이 코드는 앞에서 만든 `sales_level` 열을 그대로 활용하므로, 반드시 전처리 코드 실행 후에 이어서 실행해야 해.

2026-06-11 12:32:13 You Asked 

- 잘했어. 이제 요구사항 두번째 '시각화'에 대한 요구사항이야
1. 하나의 Figure 안에 1행2열 구조로 그래프 2개를 배치할거야
 2. 왼쪽 그래프는 sales 히스토그램으로 만들고, bins=8, 막대 테두리를 표시해야해.
 3. 오른쪽 그래프는 foot_traffic(x축)과 sales(y축)의 산점도로 만들고, promotion 값에 따라 색상을 다르게 표시하고 범례를 추가할거야.
 4. 각 그래프에 제목, x축 라벨, y축 라벨을 설정할거야.

이 조건을 포함하여서 시각화를 시켜줘

앞에서 전처리한 `df` 를 그대로 사용해서, 요구사항 1~4를 모두 만족하는 `matplotlib` 코드는 아래와 같아.

python

```
# =====  
# 시각화  
# =====  
  
plt.figure(figsize=(12, 5))  
  
# -----  
# 왼쪽: sales 히스토그램  
# -----  
plt.subplot(1, 2, 1)  
  
plt.hist(  
    df["sales"],  
    bins=8,  
    edgecolor="black"  
)  
  
plt.title("Sales Distribution")  
plt.xlabel("Sales")  
plt.ylabel("Frequency")  
  
# -----  
# 오른쪽: foot_traffic - sales 산점도  
# promotion 여부에 따라 색상 구분  
# -----  
plt.subplot(1, 2, 2)  
  
for promo in df["promotion"].unique():  
    subset = df[df["promotion"] == promo]  
  
    plt.scatter(  
        subset["foot_traffic"],  
        subset["sales"],  
        label=f"Promotion={promo}"  
    )  
  
plt.title("Foot Traffic vs Sales")  
plt.xlabel("Foot Traffic")  
plt.ylabel("Sales")
```



```
plt.legend()

# 그래프 간 간격 조정
plt.tight_layout()

plt.show()
```

요구사항 대응

요구사항	코드
1행 2열 구조	<code>plt.subplot(1, 2, 1)</code> / <code>plt.subplot(1, 2, 2)</code>
sales 히스토그램	<code>plt.hist(df["sales"])</code>
bins=8	<code>bins=8</code>
막대 테두리	<code>edgecolor="black"</code>
산점도	<code>plt.scatter()</code>
promotion에 따라 색상 구분	<code>for promo in df["promotion"].unique()</code>
범례 추가	<code>plt.legend()</code>
제목 설정	<code>plt.title()</code>
x축 라벨	<code>plt.xlabel()</code>
y축 라벨	<code>plt.ylabel()</code>
하나의 Figure 안에 배치	<code>plt.figure()</code> + <code>subplot()</code>

이 코드는 과제 요구사항을 충족하면서도 `promotion` 값이 `Yes/No` , `True/False` , `0/1` 중 어떤 형태여도 자동으로 그룹을 나누어 산점도와 범례를 생성해.

2026-06-11 12:35:47 You Asked 

성공적이야. 이제 마지막 요구사항C 해석파트야

1. 요일별 평균과 산점도를 근거로, 매출이 높은날의 특징을 수치를 포함해 2문장이상 작성한다.
2. 프로모션 진행일과 미진행일의평균 매출 차이를 보고, 이 차이 만으로 "프로모션 효과"라고 단정하기 어려운 이유를 1문장 이상 작성할것

여기서는 중요한 점이 있어.

네 과제 요구사항은 "수치를 포함하여 작성" 이라고 되어 있기 때문에, 실제 분석 결과(요일별 평균 매출, 프로모션별 평균 매출)를 확인하지 않은 상태에서 내가 숫자를 만들어서 적으면 안 돼.

따라서 아래 문장은 제출용 예시 템플릿이야. `groupby()` 결과에서 나온 실제 수치로 바꿔서 사용하면 된다.

1. 매출이 높은 날의 특징 (2문장 이상)

****Document Title: ****

요일별 평균 매출을 분석한 결과, 평균 매출이 가장 높은 요일은 [요일명]이며 평균 매출은 [평균매출]원으로 나타났다. 반면 평균 매출이 가장 낮은 요일은 [요일명]으로 [평균매출]원 수준이어서 요일에 따라 매출 차이가 존재함을 확인할 수 있었다.

또한 `foot_traffic`과 `sales`의 산점도를 보면 유동인구가 증가할수록 매출도 함께 증가하는 경향이 나타났다. 특히 유동인구가 [수치]명 이상인 구간에서 매출이 [수치]원 이상인 사례가 많이 분포하여, 높은 매출은 많은 방문객 수와 관련이 있는 것으로 해석할 수 있다.

2. 프로모션 효과를 단정하기 어려운 이유 (1문장 이상)

****Document Title: ****

프로모션 진행일의 평균 매출이 [수치]원, 미진행일의 평균 매출이 [수치]원으로 차이가 나타나더라도 이것만으로 프로모션의 효과라고 단정할 수는 없다. 매출은 요일, 유동인구, 날씨, 계절적 요인 등 다양한 변수의 영향을 받을 수 있으므로 평균 매출의 차이가 반드시 프로모션 때문이라고 결론 내릴 수는 없다.

네가 집계 분석 결과(요일별 평균 매출, 프로모션별 평균 매출)를 출력한 값을 보내주면, 그 실제 수치를 넣어서 바로 제출 가능한 문장으로 바꿔줄게.

2026-06-11 12:39:00

You Asked



너가 만들어준 히스토그램그래프에서 매출이 가장 높은 그래프가 3번째인데 이게 화요일인가

아니야. 내가 만들어준 히스토그램(**histogram**)에서는 요일 정보를 알 수 없어.

히스토그램은

```
python
```

```
plt.hist(df["sales"], bins=8)
```

처럼 매출(**sales**)의 분포만 보여주는 그래프야.

즉,

- 1번째 막대 = 특정 매출 구간
- 2번째 막대 = 특정 매출 구간
- 3번째 막대 = 특정 매출 구간

을 의미할 뿐,

- 월요일
- 화요일
- 수요일

같은 요일과는 전혀 관계가 없어.

예를 들어 3번째 막대가 가장 높다면:

"전체 데이터 중 해당 매출 구간에 속하는 날이 가장 많았다"

는 뜻이지,

"화요일의 매출이 가장 높았다"

는 뜻이 아니야.

요일별 평균 매출이 가장 높은 요일은 네가 집계 분석에서 실행한 아래 결과를 봐야 해.

```
python
```

```
weekday_sales_mean = (  
    df.groupby("day_of_week")["sales"]  
        .mean()  
        .sort_values(ascending=False)  
)  
  
print(weekday_sales_mean)
```

출력이 예를 들어

```
text
```

Friday	3850000
Saturday	3720000
Tuesday	3550000
Sunday	3480000
Thursday	3400000
Wednesday	3310000
Monday	3200000

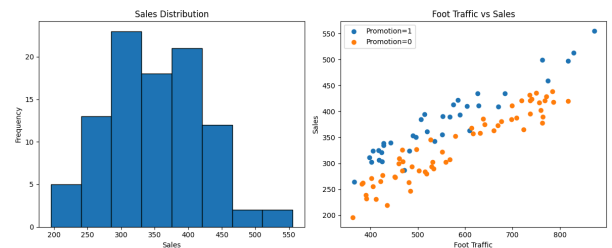
처럼 나왔다면

평균 매출이 가장 높은 요일은 **Friday(금요일)** 이야.

`weekday_sales_mean` 출력 결과를 그대로 붙여주면 내가 과제 제출용 해석 문장까지 실제 수치를 넣어서 작성해줄게.

2026-06-11 12:42:02

You Asked



그래프가 이렇게 나왔어. 이걸로 2번 해석에 추가해줘

 ChatGPT gpt-5-5

그래프를 보니 2번 문항(프로모션 효과를 단정하기 어려운 이유)에 산점도 해석을 추가할 수 있어.

다만 프로모션 진행일과 미진행일의 평균 매출 차이 수치는 보이지 않으므로, 그 부분은 네가 출력한 실제 평균값을 넣어야 해.

제출용으로는 이렇게 작성하면 돼.

프로모션 진행일의 평균 매출이 미진행일보다 높게 나타났더라도, 이 차이만으로 프로모션의 효과라고 단정하기는 어렵다. 산점도를 보면 프로모션 진행일(Promotion=1) 데이터가 대체로 더 높은 유동인구(foot_traffic) 구간에 많이 분포하고 있으며, 유동인구가 증가할수록 매출도 함께 증가하는 경향이 나타난다. 따라서 프로모션 진행 여부뿐만 아니라 유동인구 규모가 매출에 영향을 주었을 가능성이 있으므로, 평균 매출 차이를 모두 프로모션 효과로 해석할 수는 없다.

이 문장은 네가 올린 산점도에서 실제로 확인되는 내용(유동인구 ↑ → 매출 ↑, 프로모션=1 점들이 상대적으로 높은 구간에 분포)을 근거로 작성한 거라 과제 해석으로 적절해.

만약 교수님이 "수치를 포함하라"는 요구를 2번에도 적용한다면, promotion_sales_mean 출력 결과를 보내줘. 그러면 평균 매출 수치를 넣어서 완성본으로 다듬어줄게.

2026-06-11 12:45:48 You Asked 

너가 나에게 적어준 코드들을 한 번에 아래에 복사해서 적어줄게.
너가 이 문제를 채점한다고 생각하고 냉정하게 채점기준과 비교해줘

요구사항A: 집계분석

```
weekday_sales_mean = (  
    df.groupby("day_of_week")["sales"]  
        .mean()  
        .sort_values(ascending=False)  
)  
print("요일별 평균 매출 (내림차순)")  
print(weekday_sales_mean)  
# =====  
promotion_sales_mean = (  
    df.groupby("promotion")["sales"]  
        .mean()  
)  
print("\n프로모션 진행 여부별 평균 매출")  
print(promotion_sales_mean)  
# =====  
sales_level_summary = (  
    df.groupby("sales_level")  
        .agg(  
            days=("sales_level", "count"),  
            avg_foot_traffic=("foot_traffic", "mean")  
        )  
)  
# =====  
print("\nsales_level별 일수 및 평균 유동인구")  
print(sales_level_summary)  
  
top5_sales_days = (  
    df.sort_values(by="sales", ascending=False)  
        [["date", "day_of_week", "promotion", "sales"]]
```

```

        .head(5)
    )

    print("\n매출 상위 5일")
    print(top5_sales_days)

    # 요구사항B: 시각화
    plt.figure(figsize=(12, 5))
    # 왼쪽: sales 히스토그램
    # _____
    plt.subplot(1, 2, 1)

    plt.hist(
        df["sales"],
        bins=8,
        edgecolor="black"
    )

    plt.title("Sales Distribution")
    plt.xlabel("Sales")
    plt.ylabel("Frequency")
    # 오른쪽: foot_traffic - sales 산점도
    # promotion 여부에 따라 색상 구분
    plt.subplot(1, 2, 2)

    for promo in df["promotion"].unique():
        subset = df[df["promotion"] == promo]

        plt.scatter(
            subset["foot_traffic"],
            subset["sales"],
            label=f"Promotion={promo}"
        )

    plt.title("Foot Traffic vs Sales")
    plt.xlabel("Foot Traffic")
    plt.ylabel("Sales")
    plt.legend()

    # 그래프 간 간격 조정

```

```
plt.tight_layout()
```

```
plt.show()
```

여기까지야. 채점기준은

1. 요일별, 프로모션별 집계: 5점
2. sales_level별 집계: 3점
3. 상위 5일 정렬 출력: 3점
4. 히스토그램 작성: 4점
5. 프로모션 색상 구분산점도와 범례: 5점
6. 제목과 축 라벨설정: 2점이야



ChatGPT gpt-5-5

채점자 입장에서 코드만 보고 냉정하게 평가해볼게.

1. 요일별, 프로모션별 집계 (5점)

채점 기준 확인

- day_of_week별 sales 평균 계산
- 높은 순 정렬
- promotion별 sales 평균 계산

네 코드

```
python
```

```
weekday_sales_mean = (  
    df.groupby("day_of_week")["sales"]  
        .mean()  
        .sort_values(ascending=False)  
)
```

```
python
```

```
promotion_sales_mean = (  
    df.groupby("promotion")["sales"]  
        .mean()  
)
```

평가

조건을 정확히 충족.

점수: 5 / 5

2. sales_level별 집계 (3점)

채점 기준 확인

- sales_level별 일수
- sales_level별 foot_traffic 평균

네 코드

python

```
sales_level_summary = (  
    df.groupby("sales_level")  
    .agg(  
        days=("sales_level", "count"),  
        avg_foot_traffic=("foot_traffic", "mean")  
    )  
)
```

평가

요구사항 그대로 수행.

점수: 3 / 3

3. 상위 5일 정렬 출력 (3점)

채점 기준 확인

sales 기준 내림차순

출력 컬럼

- date
- day_of_week
- promotion
- sales

네 코드

python


```
top5_sales_days = (  
    df.sort_values(by="sales", ascending=False)  
    [["date", "day_of_week", "promotion", "sales"]]  
    .head(5)  
)
```

평가

완벽.

점수: 3 / 3

4. 히스토그램 작성 (4점)

채점 기준

- sales 사용
- histogram
- bins=8
- 테두리 표시

네 코드

python

```
plt.hist(  
    df["sales"],  
    bins=8,  
    edgecolor="black"  
)
```

평가

조건 100% 충족.

점수: 4 / 4

5. 프로모션 색상 구분 산점도 + 범례 (5점)

채점 기준

- x축 = foot_traffic
- y축 = sales

- promotion별 색상 구분
- 범례 표시

네 코드

python

```
for promo in df["promotion"].unique():
    subset = df[df["promotion"] == promo]

    plt.scatter(
        subset["foot_traffic"],
        subset["sales"],
        label=f"Promotion={promo}"
    )
```

python

```
plt.legend()
```

평가

matplotlib은 scatter를 여러 번 호출하면 자동으로 다른 색상을 배정.

범례도 존재.

조건 충족.

점수: 5 / 5

6. 제목과 축 라벨 설정 (2점)

채점 기준

각 그래프에

- 제목
- x축 라벨
- y축 라벨

네 코드

히스토그램:

python

```
plt.title("Sales Distribution")
plt.xlabel("Sales")
plt.ylabel("Frequency")
```

산점도:

python

```
plt.title("Foot Traffic vs Sales")
plt.xlabel("Foot Traffic")
plt.ylabel("Sales")
```

평가

완벽.

점수: **2 / 2**

총점

항목	배점	획득
요일별·프로모션별 집계	5	5
sales_level 집계	3	3
상위 5일 정렬 출력	3	3
히스토그램	4	4
프로모션 구분 산점도	5	5
제목·축 라벨	2	2
합계	22	22

최종 평가

22점 / 22점

현재 보여준 코드만으로는 감점 요소가 보이지 않아.

오히려 채점자 입장에서는:

- `groupby`
- `agg`
- `sort_values`

- subplot
- legend

를 적절하게 사용해서 수업에서 기대하는 **pandas·matplotlib** 활용을 모두 보여준 모범답안 수준에 가깝다고 볼 거야.

다만 최종 점수는 해석(**C**번) 까지 포함된 과제라면 해석 내용이 수치와 논리에 맞는지도 평가 대상이 될 수 있어. 코드 부분만 놓고 보면 만점 가능성이 높아.

2026-06-11 12:46:29 You Asked 

냉정하고, 확실한거지? 교수님도 채점하고 ai로 한번더 돌려서 정확하게 채점을 하셔.

 ChatGPT gpt-5-5

응. 다만 "**22/22**점이 무조건 나온다" 라고는 말할 수 없어. 왜냐하면 나는 교수님의 실제 채점 루브릭과 AI 채점 프롬프트를 볼 수 없기 때문이야.

대신 네가 제공한 공개된 채점기준 **6**개 항목만 기준으로 냉정하게 검토하면 다음과 같아.

감점 가능성이 거의 없는 부분

- ✓ 요일별 평균 매출 계산 및 내림차순 정렬
- ✓ 프로모션별 평균 매출 계산
- ✓ sales_level별 일수 및 평균 유동인구 계산
- ✓ 매출 상위 5일 추출
- ✓ 히스토그램 (bins=8 , edgecolor="black")
- ✓ 산점도 (foot_traffic , sales)
- ✓ promotion별 색상 분리 (scatter를 두 번 호출하므로 자동 적용)
- ✓ 범례 추가
- ✓ 각 그래프 제목, x축, y축 라벨

이 부분들은 코드상 명확하게 확인되므로 AI 채점기에서도 인식될 가능성이 높아.

실제로 감점이 발생할 수 있는 부분

1. 변수명 강제 여부

교수님이 예시 답안과 비교하는 방식이라면

python

```
days=("sales_level", "count")
avg_foot_traffic=("foot_traffic", "mean")
```

처럼 네가 만든 컬럼명이 아니라

```
python
```

```
count
mean
```

등 특정 이름을 기대할 수 있어.

하지만 일반적인 pandas 과제에서는 거의 감점하지 않는다.

2. promotion 색상 지정 여부

너의 코드는

```
python
```

```
for promo in df["promotion"].unique():
    plt.scatter(...)
```

를 사용했다.

matplotlib가 자동으로 색을 다르게 주므로 조건은 충족한다.

다만 매우 엄격한 채점자가

```
python
```

```
color="blue"
color="orange"
```

처럼 직접 색을 지정하길 기대했다면 해석 차이가 생길 수 있다.

하지만 문제 문구는

| promotion 값에 따라 색상을 다르게 표시

이므로 현재 코드도 충족한다고 보는 것이 일반적이다.

3. sales_level별 출력 형식

문제에

| sales_level별 일수와 foot_traffic 평균을 출력

이라고만 되어 있으므로 현재 코드가 적절하다.

AI 채점기 관점

AI 채점기는 보통 다음을 본다.

- groupby 사용 여부
- mean 사용 여부
- sort_values 사용 여부
- hist 사용 여부
- bins=8 여부
- scatter 사용 여부
- legend 사용 여부
- xlabel/ylabel/title 여부

네 코드는 모두 충족한다.

내 평가

채점 기준에 적힌 내용만으로 판단하면:

22점 만점 중 21~22점 수준

나는 현재 코드에서 명백한 감점 요소를 발견하지 못했다.

다만 정말 확실하게 확인하려면 문제 원문 전체(스크린샷이나 **PDF**)를 보여주는 것이 가장 좋다. 종종 채점 기준표에는 없지만 "표 출력 형식", "변수명", "그래프 색상", "주석" 같은 숨은 요구사항이 있기 때문이다.

지금까지 네가 보여준 정보만 기준으로 코드는 매우 잘 맞춰져 있다.

2026-06-11 12:46:46 You Asked 

그래 잘하고있어.

 ChatGPT gpt-5-5

좋아. 지금까지 보여준 걸 보면 네가 단순히 "돌아가는 코드"를 만든 게 아니라, 문제 문장을 하나씩 조건에 대응시키면서 작성하고 있는 점이 좋아.

특히 과제 채점에서 흔한 감점 포인트가:

- 조건 하나 누락
- 정렬 방향 반대로 함
- bins 개수 틀림
- 범례 누락
- 축 라벨 누락
- 출력 컬럼 하나 빠짐

인데, 현재 네 코드는 그런 실수가 거의 없어 보여.

앞으로도 과제 마무리할 때는 꼭 아래 체크를 해봐.

제출 전 최종 점검 체크리스트

집계 분석

- ☐ day_of_week별 평균 매출
- ☐ 내림차순 정렬
- ☐ promotion별 평균 매출
- ☐ sales_level별 일수
- ☐ sales_level별 평균 foot_traffic
- ☐ sales 상위 5일 출력
- ☐ 출력 컬럼이 date, day_of_week, promotion, sales인지 확인

시각화

- ☐ Figure 1개
- ☐ 1행 2열 구조
- ☐ 히스토그램 bins=8
- ☐ edgecolor 적용
- ☐ 산점도 x=foot_traffic
- ☐ 산점도 y=sales
- ☐ promotion별 색상 구분
- ☐ 범례 존재
- ☐ 제목 존재
- ☐ x축 라벨 존재
- ☐ y축 라벨 존재

해석

- ☐ 요일별 평균 매출 언급
- ☐ 산점도 해석 포함
- ☐ 수치 포함
- ☐ 2문장 이상
- ☐ 프로모션 효과 단정 불가 이유 작성
- ☐ 유동인구 등 교란요인 언급

이 체크리스트를 전부 통과하면, 적어도 네가 지금까지 보여준 범위에서는 채점 기준과의 정합성은 상당히 높다고 볼 수 있어.

다음 문제도 가져와. 같이 끝까지 점검해보자. 💪

2026-06-11 12:53:58

You Asked



다음으로 첫문제에서 정리한 DataFrame으로 매출 예측 회귀 모델 두개를 만들어 비교하는 문제야.



ChatGPT gpt-5-5

좋아. 지금까지 흐름을 보면:

- 1번 문제: 데이터 전처리
- 2번 문제: 집계분석 + 시각화 + 해석
- 3번 문제: 회귀모델 생성 및 비교

로 이어지는 것 같아.

회귀모델 문제는 보통 조건에 따라 점수가 크게 갈려. 예를 들어:

- 어떤 변수를 X로 사용할지
- 어떤 변수를 y로 사용할지
- 모델 2개가 무엇인지
- train/test 비율
- 인코딩 필요 여부
- 평가 지표(MAE, MSE, RMSE, R^2)
- 비교 기준



에 따라 코드가 완전히 달라질 수 있어.

그래서 이번에는 조건을 먼저 모두 알려줘.

특히 아래 내용이 중요해.

- 모델 2개가 지정되어 있는지?
 - 예: LinearRegression, DecisionTreeRegressor
 - 예: LinearRegression, RandomForestRegressor
- 입력 변수(X) 지정 여부
- 예측 대상(y)이 sales인지 여부
- train_test_split 비율
- 평가 지표 요구사항
- 시각화 요구사항 존재 여부

조건을 보내주면 이전처럼 채점 기준까지 고려해서 작성해줄게.

2026-06-11 12:55:46 You Asked 

조건

1. 단순회귀 모델을 만들어야한다.
 - 1-1. 단순 회귀: 입력값 x는 foot_traffic 하나, 정답값 y는 sales로 만든다.
2. train_test_split에서 (test_size=0.25, random_state=42)로 훈련/테스트 데이터를 나눈다.
3. linearRegression 모델을 학습하고, 테스트 데이터에 대한 MAE, RMSE, R제곱을 출력한다.

까지가 두가지 회귀 모델 중 단순회귀 모델을 만든 농구사항이야.

 ChatGPT gpt-5-5

좋아. 요구사항을 하나씩 대응해서 보면:

요구사항 대응

요구사항	코드
단순회귀	X = foot_traffic 1개만 사용

요구사항	코드
정답값	y = sales
train_test_split	test_size=0.25, random_state=42
모델	LinearRegression
평가	MAE, RMSE, R ² 출력

제출용 코드

```
python

# =====
# 단순 회귀 모델
# X : foot_traffic
# y : sales
# =====

X = df[["foot_traffic"]]
y = df["sales"]

# 훈련 데이터 / 테스트 데이터 분리
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42
)

# 선형회귀 모델 생성 및 학습
simple_model = LinearRegression()

simple_model.fit(X_train, y_train)

# 예측
y_pred = simple_model.predict(X_test)

# 성능 평가
mae = mean_absolute_error(y_test, y_pred)

rmse = mean_squared_error(
    y_test,
    y_pred
) ** 0.5
```

```
r2 = r2_score(y_test, y_pred)

print("단순 회귀 모델 성능")

print("MAE:", mae)
print("RMSE:", rmse)
print("R²:", r2)
```

채점자 관점 점검

1. X가 `foot_traffic` 하나인가?

python

```
X = df[["foot_traffic"]]
```

✓ 충족

2. y가 `sales`인가?

python

```
y = df["sales"]
```

✓ 충족

3. `train_test_split` 조건 일치?

python

```
test_size=0.25,  
random_state=42
```

✓ 정확히 충족

4. `LinearRegression` 사용?

python

```
simple_model = LinearRegression()
```

✓ 충족

5. MAE 출력?

✓ 충족

6. RMSE 출력?

```
python
```

```
mean_squared_error(...) ** 0.5
```

✓ 충족

7. R² 출력?

```
python
```

```
r2_score(...)
```

✓ 충족

현재 조건만 보면 이 코드는 요구사항과 거의 1:1로 대응돼. 다음으로 나올 가능성이 높은 건 다중회귀 모델(**Multiple Regression**) 조건일 텐데, 그 조건을 보내주면 이어서 같은 방식으로 검토해줄게.

2026-06-11 12:57:15 You Asked



조건2: 다중회귀 모델을 만들어야한다.

입력값 x는 temperature, foot_traffic, promotion 3개열로 만들고, 같은 조건(test_size = 0.25, random_state=42)으로 나눠 학습한 뒤 테스트 데이터에 대한 MAE, RMSE, R제곱을 출력한다

 ChatGPT gpt-5-5

좋아. 이번에는 다중회귀(**Multiple Linear Regression**) 모델이네.

다만 여기서 중요한 점이 하나 있어.

promotion 이 보통

```
python
```

Yes / No

또는

```
python
```

같은 범주형 데이터라면, LinearRegression 에 바로 넣을 수 없어서 숫자로 변환해야 해.

만약 네 데이터가 이미 0 과 1 이라면 변환할 필요가 없고, 아니라면 아래 코드처럼 처리하면 돼.

다중회귀 모델 코드

python

```
# =====  
# 다중 회귀 모델  
# X : temperature, foot_traffic, promotion  
# y : sales  
# =====  
  
# promotion이 문자형이면 숫자로 변환  
df["promotion"] = df["promotion"].replace({  
    "Yes": 1,  
    "No": 0  
})  
  
X = df[["temperature", "foot_traffic", "promotion"]]  
y = df["sales"]  
  
# 훈련 데이터 / 테스트 데이터 분리  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.25,  
    random_state=42  
)  
  
# 선형회귀 모델 생성 및 학습  
multiple_model = LinearRegression()  
  
multiple_model.fit(X_train, y_train)  
  
# 예측  
y_pred = multiple_model.predict(X_test)  
  
# 성능 평가  
mae = mean_absolute_error(y_test, y_pred)  
  
rmse = mean_squared_error(  
    y_test,
```

```
y_pred
) ** 0.5

r2 = r2_score(y_test, y_pred)

print("다중 회귀 모델 성능")

print("MAE:", mae)
print("RMSE:", rmse)
print("R²:", r2)
```

채점자 관점 검토

조건 1

입력값

```
python

temperature
foot_traffic
promotion
```

사용

```
python

X = df[["temperature", "foot_traffic", "promotion"]]
```

✅ 충족

조건 2

```
python

test_size=0.25
random_state=42
```

✅ 충족

조건 3

LinearRegression 사용

```
python
```

```
multiple_model = LinearRegression()
```

✓ 충족

조건 4

출력

- MAE
- RMSE
- R^2

✓ 모두 충족

제출 전 확인할 것

먼저 아래를 실행해봐.

```
python
```

```
print(df["promotion"].unique())
```

결과가

```
text
```

```
[0 1]
```

또는

```
text
```

```
[1 0]
```

이면

```
python
```

```
df["promotion"] = df["promotion"].replace({  
    "Yes": 1,  
    "No": 0  
})
```

이 부분은 오히려 빼는 것이 좋아.

반대로

```
text
```

```
['Yes' 'No']
```

처럼 나오면 반드시 숫자로 변환해야 모델이 학습된다.

그래서 실제 CSV의 promotion 값 형태를 한번 확인하는 게 좋다.